

09

/ 13 LECTURES

DNS and Internet Applications

The Internet's phone book and its favourite protocol: DNS hierarchy and resolution, URL anatomy, HTTP requests and responses, status codes, cookies and sessions.

Why DNS

Root · TLD · Authoritative

Recursive Resolver

Caching & TTL

URL Anatomy

HTTP Request/Response

Methods

Status Codes

Cookies

Sessions

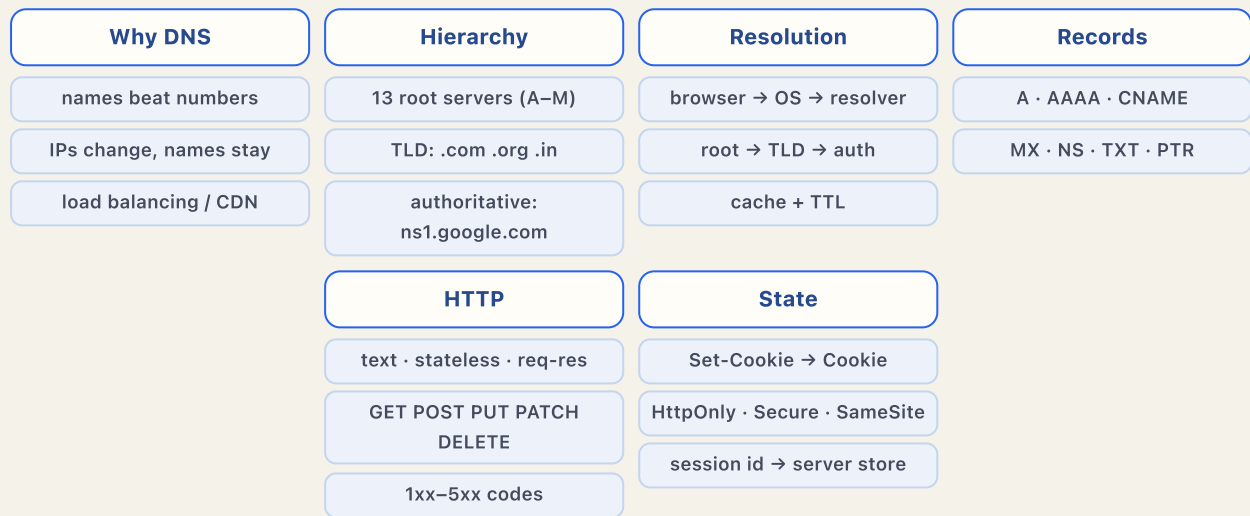
WHAT THIS LECTURE GIVES YOU

Everything between "typing a name" and "seeing a page": why DNS exists, its three-tier hierarchy, the full 9-step resolution walk, record types and dig commands; then HTTP — message formats byte-by-byte, methods with idempotency/safety, status-code families, and how cookies + sessions put memory into a stateless protocol.

LEARNING OBJECTIVES — AFTER THIS LECTURE YOU CAN

- Explain **why DNS** exists and walk its **hierarchy** (root → TLD → authoritative)
- Trace the **full resolution sequence** with caching and TTL
- Interpret **DNS record types** and use `dig` / `nslookup`
- Parse a **URL** into all seven components
- Read/write **HTTP requests & responses**; know methods and **status codes**
- Explain **cookies vs sessions** and their security flags

DNS & HTTP



1 Why DNS?

DNS (Domain Name System) translates human-readable names (`google.com`) into IP addresses (`142.250.195.46`). Why not use IPs directly?

- **Memory:** names beat dotted quads (especially IPv6 monsters)
- **Stability:** servers change IPs; the name stays constant
- **Scaling:** one name → many IPs (load balancing) · many users → nearest IP (CDN geo-routing)

ANALOGY DNS = the Internet's **phone book**: you know the name ("Mom"), the phone book gives the number. And like a good contact app, everyone **caches** recently-dialed numbers so the book isn't opened every time.

2 The DNS Hierarchy

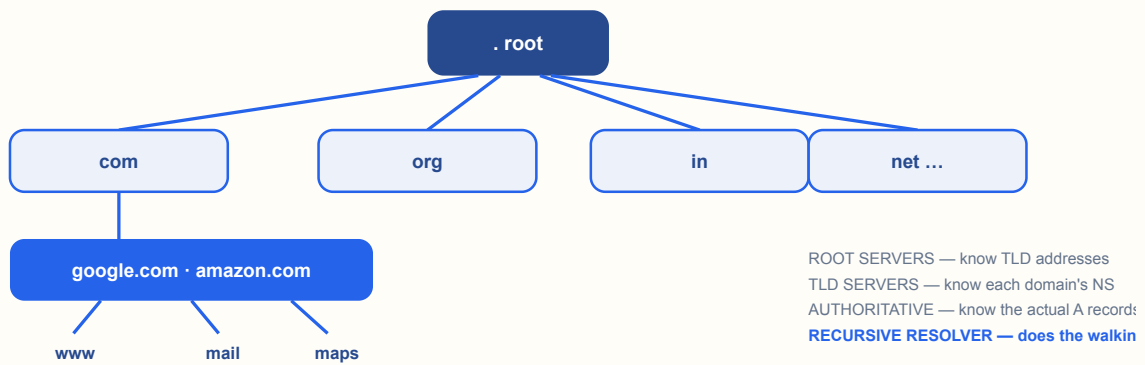


Figure 1. A distributed tree: no single server knows everything — each level delegates downward.

Server type	Knows	Example
Root servers	Which servers run each TLD (13 logical identities A–M)	Operated by ICANN, Verisign...
TLD servers	The authoritative nameservers of every domain under their TLD	Verisign runs .com
Authoritative servers	The actual records for one specific domain	ns1.google.com
Recursive resolver	Nothing by default — it <i>walks the tree for you</i> and caches answers	Your ISP's DNS · 8.8.8.8 · 1.1.1.1

3 Full Resolution — Step by Step

Query: browser needs the IP for `www.google.com`

- 1 Browser cache**
Fresh cached answer? Use it. Done — no network.
- 2 OS cache + /etc/hosts**
Local system-level cache and hosts file.
- 3 Ask the recursive resolver**
Query to configured DNS (router, ISP, 8.8.8.8) — UDP port 53.
- 4 Resolver cache**
Cached & TTL-valid? Instant answer. Else start walking.
- 5 Resolver → ROOT server**
Root: "Not me — the **.com** TLD is at 192.5.6.30." (**referral**)
- 6 Resolver → .com TLD**
TLD: "Ask Google's nameservers — **ns1.google.com.**" (**referral**)
- 7 Resolver → authoritative**
ns1: "**www.google.com = 142.250.195.46**, TTL 300" (**answer**)

8-9 Cache + return

Resolver caches for TTL seconds and returns the IP; browser connects to it.

EXAM **Iterative vs recursive:** the *resolver* does iterative walks (ask root, then TLD, then auth); your machine makes one *recursive* request ("get me the answer, however many steps it takes"). Draw the 9 arrows — it's a guaranteed diagram question.

Caching & TTL

- Every record carries a **TTL (seconds to cache)**; after expiry the resolver must re-ask.
- **Low TTL** = fast changes, more queries · **High TTL** = fewer queries, slow propagation.

4 DNS Record Types & Lookup Tools

Record	Purpose	Example
A	name → IPv4	google.com → 142.250.195.46
AAAA	name → IPv6	google.com → 2607:f8b0::200e
CNAME	alias → canonical name	www.example.com → example.com
MX	mail servers for the domain	google.com MX → smtp.google.com
NS	the domain's nameservers	google.com NS → ns1.google.com
TXT	arbitrary text — SPF, DKIM, verification	"v=spf1 include:_spf.google.com ~all"
PTR	reverse: IP → name	46.195.250.142... → google.com

DNS LOOKUP COMMANDS

TERMINAL

```
dig www.google.com           # basic A lookup
dig @8.8.8.8 www.google.com  # ask a specific resolver
dig +trace www.google.com    # walk root-TLD-auth live!
dig google.com MX · AAAA · NS · TXT # specific record types
dig -x 142.250.195.46        # reverse (PTR) lookup
nslookup -type=MX google.com # the simpler cousin
```

```
# reading dig output:
;; ANSWER SECTION:
www.google.com.  300  IN  A  142.250.195.46
#                ↑TTL      ↑type ↑ answer
```

5 URL Anatomy

https:// scheme	user:pass@ userinfo (rare)	www.example.com host (DNS)	:8080 port	/path/to/resource path	? q=search&page=2 query	#section3 fragment
--------------------	-------------------------------	-------------------------------	---------------	---------------------------	-------------------------------	-----------------------

Figure 2. Every piece of a URL — defaults: port 80 (HTTP) / 443 (HTTPS) omitted.

TRAP

The **fragment never leaves the browser** — servers never see #section3. Query strings ARE sent and must be URL-encoded. Favourite MCQ.

6 HTTP — Request & Response

HTTP is **text-based** (human-readable), **stateless** (each request is independent), and **request-response** (client initiates, server responds).

HTTP REQUEST CLIENT → SERVER

```
GET /products?id=42 HTTP/1.1 ← request line
Host: www.example.com ← required in 1.1
User-Agent: Mozilla/5.0
Accept: text/html
Cookie: session=abc123 ← blank line
{ json body — POST/PUT only }
```

HTTP RESPONSE

SERVER → CLIENT

```
HTTP/1.1 200 OK ← status line
Content-Type: text/html; charset=UTF-8
Content-Length: 4523
Set-Cookie: session=abc; HttpOnly; Secure
Cache-Control: max-age=3600 ← blank line
<!DOCTYPE html> <html>... ← body
```

Methods

Method	Purpose	Body?	Idempotent?	Safe?
GET	Retrieve	No	Yes	Yes
POST	Create / submit	Yes	No	No
PUT	Replace whole resource	Yes	Yes	No
PATCH	Partial update	Yes	No	No
DELETE	Remove	No	Yes	No
HEAD	Headers only	No	Yes	Yes
OPTIONS	What's allowed?	No	Yes	Yes

Definitions + REST pattern

Idempotent = repeating gives the same result as once. **Safe** = changes nothing. REST: GET /api/users list · POST /api/users create · GET /api/users/42 one · PUT /api/users/42 replace · PATCH partial · DELETE remove.

Status codes

Family	Meaning	Know cold
1xx	Informational	100 Continue
2xx	Success	200 OK · 201 Created · 204 No Content
3xx	Redirection	301 permanent · 302 temporary · 304 Not Modified
4xx	Client error	400 Bad Request · 401 Unauthorized · 403 Forbidden · 404 Not Found · 429 Too Many
5xx	Server error	500 Internal · 502 Bad Gateway · 503 Unavailable · 504 Timeout

301 vs 302

permanent (browsers & SEO update) vs temporary redirect.

401 vs 403

401 "who are you?" (not authenticated) vs 403 "I know you — still no" (not authorized).

500 vs 502

this server broke vs the upstream server sent garbage (gateway view).

7 Cookies & Sessions — Memory for a Stateless Protocol

HTTP forgets you between requests. **Cookies** fix it: the server hands the browser a token; the browser returns it on every subsequent request — the server recognises the token and "remembers".

1 Login POST → 2 Set-Cookie: session=abc → 3 Browser stores →

4 Cookie: session=abc (every request) → 5 Server maps id → your session

Attribute	Effect
HttpOnly	JavaScript can't read the cookie → blunts XSS token theft
Secure	Sent over HTTPS only → no plaintext interception
SameSite=Strict	Never sent on cross-site requests → CSRF defence
Max-Age=N	Expires after N seconds
Domain=.example.com	Shared across all subdomains

	Cookie (client-side)	Session (server-side)
Stored at	Browser	Server (DB / Redis)
Contains	Session ID (or data)	The actual user data
Size	~4 KB	Unlimited
Visible to user?	Yes	No

EXAM

Authentication vs Authorization: authN = proving identity (login, 401 when missing); authZ = checking permission (403 when denied). And: servers store a **session ID** in the cookie — not your data — so a stolen cookie is a stolen identity but a leaked DB row doesn't leak sessions.

LECTURE 9 — RAPID REVISION CARD

DNS: name → IP; hierarchy root → TLD → authoritative; resolver walks it for you.

Records: A · AAAA · CNAME alias · MX mail · NS · TXT · PTR reverse.

URL = scheme://user@host:port/path?query#fragment
— **fragment stays in browser.**

Methods: GET safe+idempotent · POST neither · PUT idempotent · PATCH partial.

Cookies: Set-Cookie → auto-returned; HttpOnly (XSS) + Secure (HTTPS) + SameSite (CSRF).

Resolution: browser → OS → resolver cache → root → TLD → auth → cached (TTL).

dig +trace shows the whole walk; TTL controls cache freshness.

HTTP: text, stateless, request-response; Host header required in 1.1.

Codes: 2xx ok · 3xx redirect (301 perm / 302 temp / 304 cached) · 4xx your fault (401 vs 403) · 5xx server's fault.

Session: ID in cookie, data on server.

✦ Important Questions — Exam Practice

A. Multiple Choice

Q1. Which DNS record maps a domain to an IPv6 address?

MCQ

- a) A
- b) CNAME
- c) AAAA
- d) MX

Q2. In DNS resolution, a "referral" is:

MCQ

- a) the final IP answer
- b) a pointer to the next server to ask
- c) a cached entry
- d) an error

Q3. Which part of the URL is never sent to the server?

MCQ

- a) path
- b) query string
- c) host
- d) fragment (#)

Q4. 403 Forbidden means:

MCQ

- a) you aren't authenticated
- b) you're authenticated but not allowed
- c) server crashed
- d) resource moved

Q5. Which HTTP method is NOT idempotent?

MCQ

- a) GET
- b) PUT
- c) DELETE
- d) POST

Q6. The HttpOnly cookie attribute protects against:

MCQ

- a) CSRF
- b) XSS stealing the cookie
- c) eavesdropping
- d) session expiry

B. Short Answer (2–4 marks)

Q7. List the order of caches consulted before any DNS server is contacted.

SHORT

Q8. What is a CNAME and when would you use one?

SHORT

Q9. Differentiate 301 vs 302 and 401 vs 403.

SHORT

Q10. Why store only a session ID in the cookie instead of the user's data?

SHORT

Q11. Which DNS record type configures email for a domain, and which proves domain ownership to a CA?

SHORT

C. Long Answer (5–8 marks)

Q12. Draw and explain the complete DNS resolution for `www.google.com`: every cache, server type, referral and answer, and where TTL applies. Distinguish recursive vs iterative queries.

LONG

Q13. Explain the HTTP request-response cycle with annotated request and response messages. Cover methods (idempotent/safe), status-code families with examples, and keep-alive.

LONG

Q14. How do cookies and sessions create state over stateless HTTP? Include the Set-Cookie flow diagram and explain HttpOnly, Secure, and SameSite=Strict with the attack each defeats.

LONG

Q15. Explain URL anatomy with the full annotated example; which components go to the server and which stay local?

LONG

ANSWER KEY

A1. c — AAAA ("quad-A").

A2. b — "I don't know, ask X" (root→TLD, TLD→auth).

A3. d — the fragment is browser-only.

A4. b — authenticated, not authorized.

A5. d — POST repeats create duplicates.

A6. b — JS can't read it; CSRF is SameSite's job; eavesdropping is Secure's.

A7. Browser cache → OS resolver cache → /etc/hosts → (then the recursive resolver's own cache).

A8. Alias pointing to a canonical name — e.g. www → root domain, or pointing to a CDN.

A9. 301 permanent (SEO updates) vs 302 temporary; 401 unauthenticated "who are you" vs 403 unauthorized "not allowed".

A10. 4 KB limit, tampering by client, data exposure — the ID is a capability token, real data stays server-side.

A11. MX for mail routing; TXT for ownership/SPF/DKIM proofs.

A12–A15. Reproduce the 9-step timeline, Figure 1, the request/response code blocks, and the cookie flow — annotated diagrams earn the marks.

Wrap-Up — carry these into Lecture 10

NAME → IP Stub resolver → recursive resolver → **root** → **TLD** → **authoritative**, cached at every step. UDP 53 (TCP 53 for big/zone answers).

RECORDS A name→IPv4 · AAAA →IPv6 · **CNAME** alias · **MX** mail · **TXT** proofs · **NS** delegation.

HTTP SHAPES GET/PUT/DELETE are **idempotent**; POST is not. 301/302 redirect, 401 vs 403, cookies carry identity (4 KB, credential-ish).

NEVER SENT The **#fragment** stays in the browser; query strings go to the server and must be URL-encoded.

NEXT **Lecture 10 — TCP, UDP & Sockets:** the transport layer beneath it all — ports, the 3-way handshake, reliability, and your first socket code.